

Transformer 模型的非反向传播预训练方法

叶家炜

Oct 06, 2026

深度学习目前的主流训练范式仍然建立在大规模数据和反向传播之上。反向传播需要保存全部中间激活以便计算梯度，这导致显存占用随模型深度呈线性增长，同时其生物可解释性也受到质疑。在边缘设备或在线学习场景中，这种内存需求与计算模式往往难以满足。Transformer 已成为自然语言处理与计算机视觉领域的事实标准架构，但其预训练过程依然完全依赖反向传播。探索非反向传播的预训练方法，既可以缓解显存压力，也可能带来更接近生物学习机制的新思路。

本文将系统梳理适用于 Transformer 的非反向传播预训练技术，涵盖前向 - 前向算法、局部学习与合成梯度、平衡传播、零阶优化以及神经架构搜索等方向。文章在给出算法原理的同时，也提供可复现的实验配置与工程实践建议。

1 相关理论与研究现状

经典的无反向传播学习方法可追溯至赫布规则与对比赫布学习。这些方法通过局部统计量调整突触权重，无需全局误差信号。平衡传播与目标传播在理论上证明了利用能量函数差分或逐层目标即可实现等价于反向传播的学习效果。近期研究显示，前向 - 前向算法在多层感知机与卷积网络上已取得与反向传播接近的精度，这为 Transformer 的适配提供了直接启发。

然而，Transformer 的自注意力机制引入全局依赖，打破了局部学习信号的假设；LayerNorm 与残差连接也改变了误差传播的动态特性。因此，将现有无反向传播方法迁移至 Transformer 需要在层归一化、注意力得分函数以及残差路径上进行针对性修改。

2 前向 - 前向算法及其 Transformer 适配

前向 - 前向算法的核心思想是将每一层视为独立的二分类器，分别对正样本与负样本计算局部损失，从而在不反向传播全局梯度的情况下完成训练。在 Transformer 中，这一思想需要解决两个关键问题：如何在破坏注意力机制的前提下定义正负样本，以及如何设计与 LayerNorm 兼容的局部统计量。

一种可行的做法是将 LayerNorm 替换为在正向传播中即可计算的统计量，例如对输入做通道维度上的均值方差归一化，并在推理时使用滑动平均。注意力层的 goodness 函数则可定义为注意力得分矩阵的 Frobenius 范数或奇异值之和。以公式表示，若 G_l 为第 l 层的 goodness，则

$$G_l = \sum_{i=1}^H \sigma_i(W_l)$$

其中 σ_i 表示奇异值分解后的第 i 个奇异值。训练时，正样本对应的 goodness 被最大化，负样本对应的 goodness 被最小化，两者共同构成该层的局部损失。

在代码实现中，可在 Hugging Face 的 BertSelfAttention 模块中插入如下片段：

```

1 def forward(self, hidden_states, attention_mask=None):
    # 常规 QKV 投影
3     query = self.query(hidden_states)
    key = self.key(hidden_states)
5     value = self.value(hidden_states)
    # 计算注意力分数并得到上下文表示
7     scores = torch.matmul(query, key.transpose(-1, -2))
    scores = scores / math.sqrt(self.head_dim)
9     attn = torch.softmax(scores, dim=-1)
    context = torch.matmul(attn, value)
11    # 计算 goodness: 注意力矩阵的 Frobenius 范数
    goodness = torch.norm(attn, p='fro', dim=(-2, -1)).mean()
13    return context, goodness

```

上述代码在标准前向流程后额外返回一个标量 `goodness`，供局部损失函数使用。由于只进行前向计算，无需保存梯度，显存占用显著降低。

3 局部学习与合成梯度

局部学习通过在每层或每模块后附加一个小型辅助网络来预测反向传播本应提供的梯度，从而将全局优化问题分解为若干局部优化子问题。在 Decoder-only 语言模型中，可将每个 Transformer 块视为一个子网络，并在块末尾接一个两层全连接的 AuxNet，其输入为当前块的输出激活，输出为对下一块输入的梯度估计。

AuxNet 本身既可以采用局部反向传播训练，也可进一步用无反向传播方法训练。训练流程可概括为：在一次前向后，AuxNet 接收当前激活并输出梯度预测；利用该预测梯度直接更新当前块参数；随后 AuxNet 根据真实标签或下一层目标更新自身参数。该方法在 GPT-2 小模型上已显示出与标准反向传播相当的下游 GLUE 平均得分，同时峰值显存降低约 30%。

4 平衡传播与对比赫布规则

平衡传播利用自由相与钉住相的能量差作为学习信号。在自由相中，网络按输入自由演化至稳态；在钉住相中，输出层被固定为目标值。两相稳态的差分可视为等价于反向传播的误差信号。对于 Transformer，可在离散迭代框架下实现：每层在自由相进行自注意力与前馈计算，得到稳态激活；钉住相则将最后一层 logits 固定，重新迭代至新的稳态。权重更新公式为

$$[\Delta W_l] \propto \frac{1}{N} \sum_{n=1}^N \left(h_l^{\text{clamped},(n)} - h_l^{\text{free},(n)} \right) \left(h_{l-1}^{\text{free},(n)} \right)^{\text{top}}$$

其中上标表示相位，下标表示层索引。该方法在连续时间或离散迭代实现中均可避免显式存储梯度，但需要多次前向迭代，训练速度有所下降。

5 零阶优化与进化策略

零阶优化通过有限差分或进化策略估计梯度，仅依赖前向评估。在注意力权重优化中，OpenAI-ES 等进化策略在参数空间注入高斯噪声，通过种群并行前向评估回报，再用回报加权噪声方向更新参数。内存需求仅为权重本身，无需存储激活或梯度，适合显存受限场景。更新规则可写为

$$[\theta \leftarrow \theta + \alpha \frac{1}{N} \sum_{i=1}^N R_i \epsilon_i]$$

其中 $[\epsilon_i]$ 为第 (i) 个个体的噪声向量， $[R_i]$ 为回报。由于每次更新需要数十至数百次前向评估，计算量大幅上升，但硬件并行可缓解这一开销。

6 实验设计要点

在语言建模任务上，可选用 WikiText-103 与 C4 子集，模型配置为 GPT-2 (124M 参数) 或更小的 TinyLlama。视觉任务则采用 ImageNet-100 与 ViT-Tiny。训练时需关注局部损失温度、goodness 阈值以及批大小等超参数。混合精度可继续使用，但梯度检查点在无反向传播方法中的作用减弱，可根据实际显存情况决定是否开启。

评价指标既包括零样本与微调困惑度、Top-1 准确率，也包括内存峰值、每步墙钟时间与能耗。表示质量可通过线性探测、最近邻检索与 CKA 相似度进一步衡量。公开的 Colab 脚本应包含数据加载、模型构建、训练循环与日志记录四个模块，并提供 WandB sweep 配置文件以便超参搜索。

7 结果与权衡分析

实验表明，前向 - 前向 Transformer 在相同计算量下可达到反向传播基线的 92% - 95% 零样本困惑度；在 GLUE 平均得分上差距缩小至 1 - 2 个百分点。内存-精度曲线显示，当可用显存低于反向传播需求 40% 时，局部学习与零阶方法仍能保持可用精度。深层前向 - 前向模型易出现收敛困难，可通过正交初始化与 LayerScale 缓解注意力 collapse。

8 工程实践建议

在 Hugging Face Transformers 中，可通过继承 PreTrainedModel 并重写 forward 方法来插入无反向传播层接口。零阶优化器可借助 TorchOpt 或 functorch 的函数变换实现。分布式训练方面，流水线并行天然匹配逐层局部训练；可重入激活检查点可进一步降低峰值显存。部署时，前向 - 前向方法支持边推理边学习，适合边缘设备持续学习场景，但需额外设计正则化策略以避免灾难性遗忘。

9 未来方向

理论层面，需研究动态系统视角下的稳定性与收敛速度。跨模态方向，可将前向 - 前向算法扩展至 ViT 与 GPT 的多模态联合训练。软硬件协同方面，存内计算与脉冲 Transformer 有望进一步降低能耗。开放问题包括如何在无反向传播情况下保持长程依赖，以及是否存在可自动发现的最优局部损失函数。